

자료구조 기말고사

10주차 Red Black Tree

Red Black Tree

A binary search tree in which every node is colored with either Red or Black
모든 node가 Red 또는 Black으로 색칠된 binary search tree

Binary Tree : a tree where every node has 0, 1, or 2 children.

Each child node is labeled as either a left child or a right child.

모든 node가 0, 1, 또는 2개의 children을 가지는 tree

각 child node는 left child 또는 right child로 label된

Binary Search Tree : Binary Search Tree is a binary tree, in which for every node, the **right** subtree contains only nodes with values **larger than the node's value**, and **left** subtree contains only nodes with values **smaller than the node's value**.

이진 트리는 운 나쁘면(편향되면) $\Theta(n)$, 운 좋으면 $\Theta(\log n)$ -> 우리는 $\Theta(\log n)$ 을 보장하고자 한다.

Red-Black Properties

1. Every node is either Red or Black.

모든 node는 Red 또는 Black이다.

2. The root is always Black.

root는 항상 Black이다.

3. Every leaf (NULL=NIL) is Black.

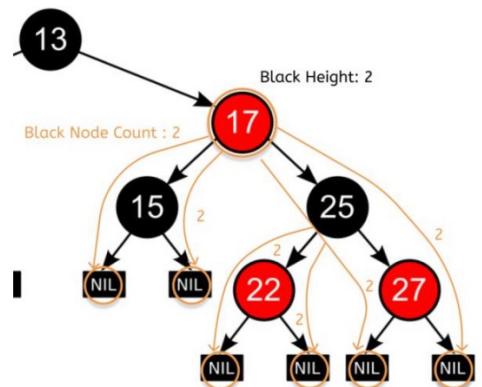
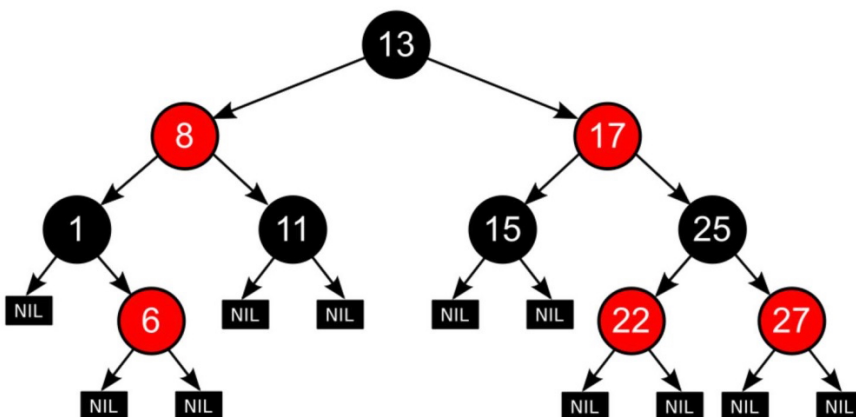
모든 leaf (NULL=NIL)는 Black이다.

4. If a node is **Red**, both its children are **Black** (a Black node can have both Red/Black children).

만약 어떤 node가 Red라면, 그것의 두 children은 모두 Black이다 (Black node는 Red/Black children을 모두 가질 수 있다).

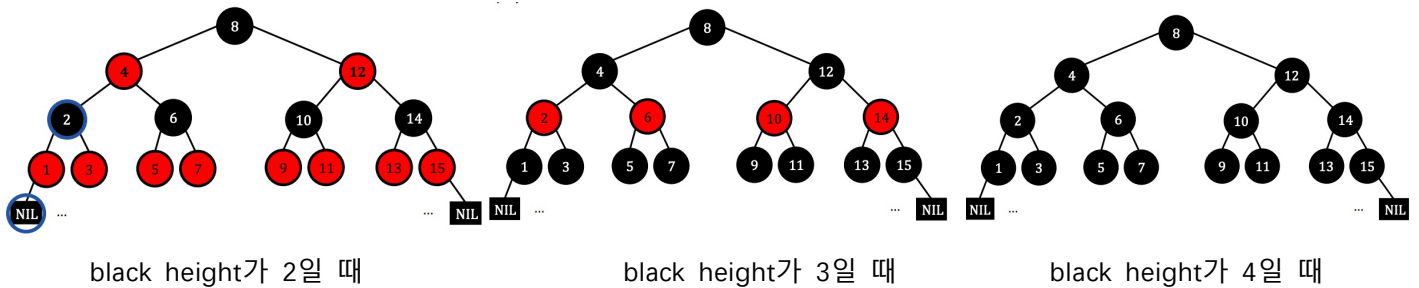
5. For each node, all paths from a node A to a NIL node B contain the same number of Black nodes.

각 node에 대해, 특정 node A에서 NIL node B까지의 모든 path는 동일한 수의 Black node를 포함한다. (단, 자기 자신은 포함 X)



Given a complete binary tree of height 3 with the key {1,2,3,4,...,15}, add the NIL leaves and color the nodes in three different ways such that the black heights of root '8' is 2, 3, and 4.

{1,2,3,4,...,15}의 key를 가진 height 3인 complete binary tree가 주어졌을 때, NIL leaves를 추가하고 root '8'의 black height가 2, 3, 4가 되도록 세 가지 다른 방법으로 node들을 색칠하라.



Balancing하기

insert나 delete를 하다보면 red-black properties를 잃을 수 있다.

-> 1. recoloring 2. rotation 두 종류의 조치를 해야 한다.

Inserting하기

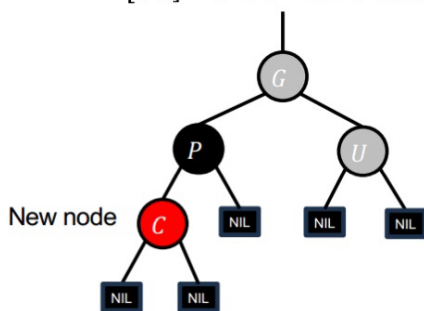
- [1] Perform standard BST insertion and make the color of new node as **Red**
- [2] If parent is **Black**, then do nothing!
- [3] If parent is **Red**, then do the following:
 - [3-a] If 'uncle' node U is **Red**, then '**Recolor**'
 - [3-b] If 'uncle' node U is **Black**, then '**Rotate+ Recolor**'

신규 노드는 **Red**

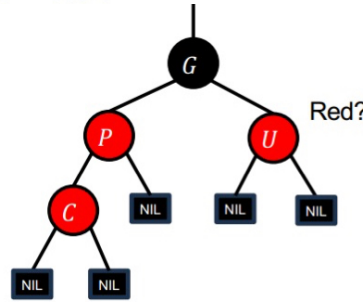
부모가 **Red**면 조치를 해야 함

Uncle이 **Red**면 Recolor

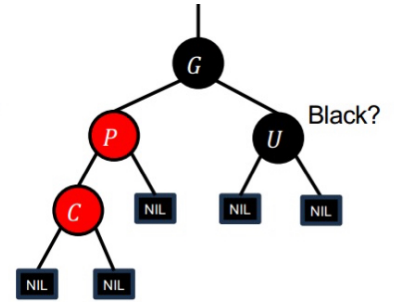
Uncle이 **Black**이면 Rotate+Recolor



(2): the parent is black



(3)-(a): the uncle is red



(3)-(b): the uncle is black

[3-a] 부모가 **Red**인데 Uncle이 **Red**일 때

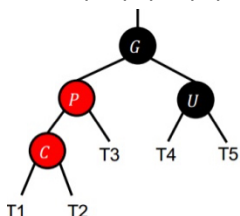
-> 부모 노드와 삼촌 노드의 색상을 모두 Black으로 변경하고, 조부모 노드를 Red로 변경

만약 조부모의 부모 역시 Red였다면 다시 한번 위반이 발생하게 됩니다. 이 경우에는 바뀐 조부모 노드를 새로 삽입된 노드처럼 취급하여 트리의 상층부로 올라가며 똑같은 원리로 위반 사항을 계속 해결해야 함

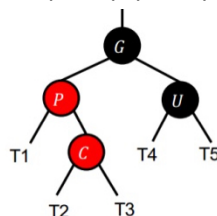
[3-b] 부모가 **Red**인데 Uncle이 **Black**일 때

방향에 따라 시나리오를 구분해야 한다.

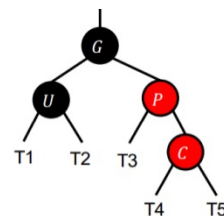
LL LR RR RL Cases



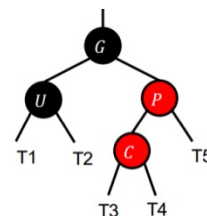
LL Case



LR Case

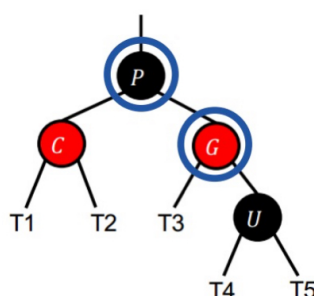
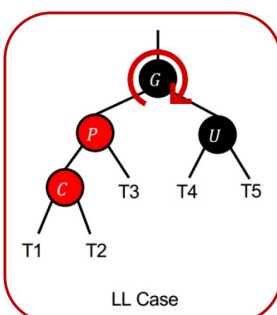


RR Case



RL Case

LL일 때

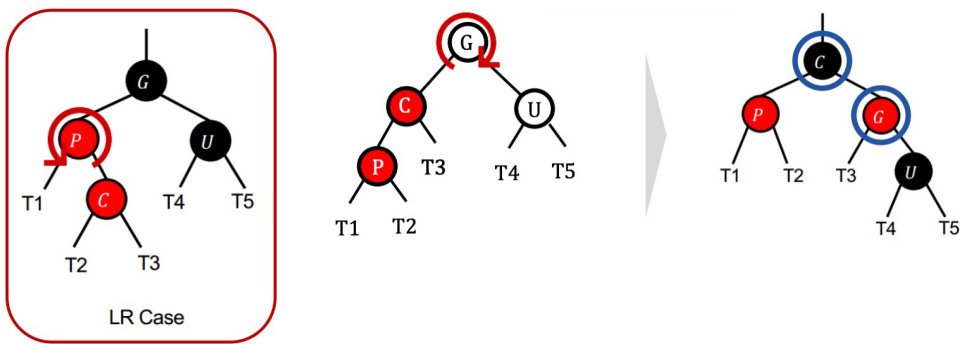


부모를 **Black**으로, 조부모를 **Red**로 색상변경

그다음 Right Rotation

Subtree는 BST 특성에 맞게 잘 매달아 줘야 함

LR일 때



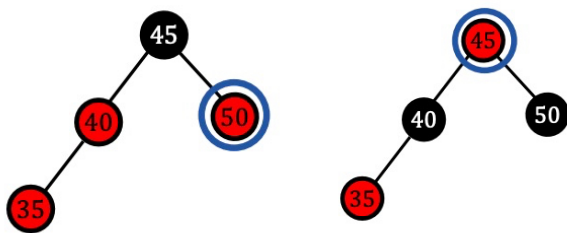
부모 노드를 기준으로 좌회전
 그다음 부모를 Black으로, 조부모를 Red로.
 조부모를 축으로 우회전
 Subtree는 BST 특성에 맞게 잘 매달아 줘야 함

e.g. [3-a] 에서 35가 들어왔다.

Uncle이 Red네?

Parent, Uncle를 Black,

조부모를 Red로 하자



그런데 Root(45)가 Red네? -> Black으로 칠해주자

Deleting

- [1] If the node has no child? Simply remove it!
- [2] If the node has a single child? Replace the node with its child!
- [3] If the node has two children? Replace the node with its successor!

자식이 없으면 그냥 지우기

자식 1개면? 그 자식으로 대체하기

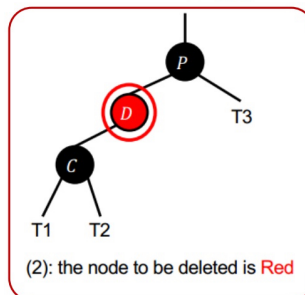
자식이 2개면? successor로 대체하기

Red 노드 삭제하기

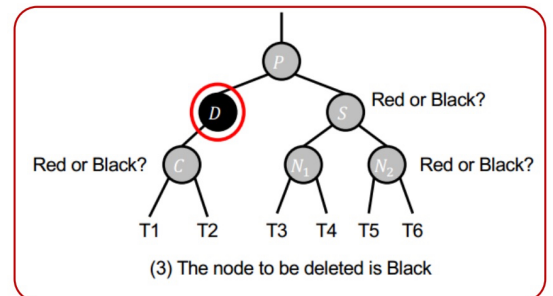
그냥 삭제하기

자식 2개면 최솟값을 찾아서 올리기

No violation if D is Red



(2): the node to be deleted is Red

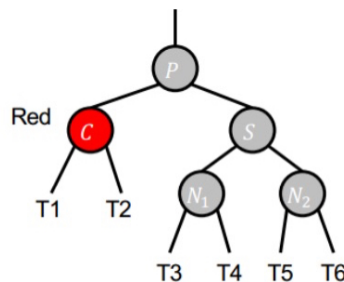
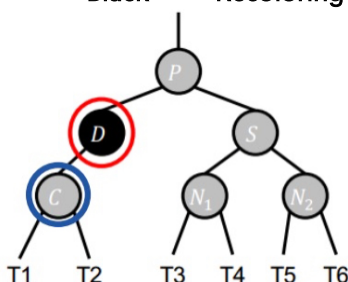


(3) The node to be deleted is Black

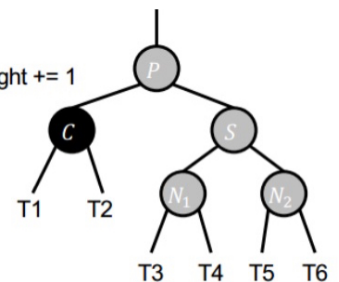
Black 노드 삭제하기

Case 1. Successor가 Red일 때

-> Black으로 Recoloring



Black height += 1



Case 2. Successor가 Black일 때

-> Rotation하고 Recoloring

4가지 Case가 있다

Case 2-1 후계자가 Black이고 Sibling(형제)가 Red일 때

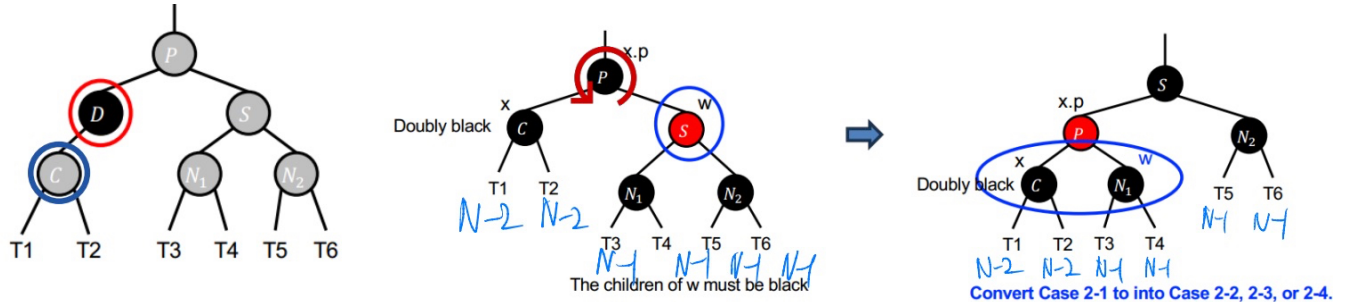
-> Left Rotation + Recoloring

원래 균형이 맞던 트리에서 BLACK 노드(D)를 하나 싹둑 지워버리면, 그 노드가 지나가던 경로의 BLACK 개수가 다른 곳보다 1개 부족(n-1)해집니다.

그래서 Doubly Black 문제가 발생했기 때문에 해결을 해 줘야 한다.

대원칙 : 삭제 연산을 수행할 때 형제(Sibling) 노드는 절대로 Red여서는 안 된다

-> 형제 노드를 BLACK 형태로 되돌려놓아야. 형제가 Red인 상태에서는 삭제 문제 직접 해결 불가



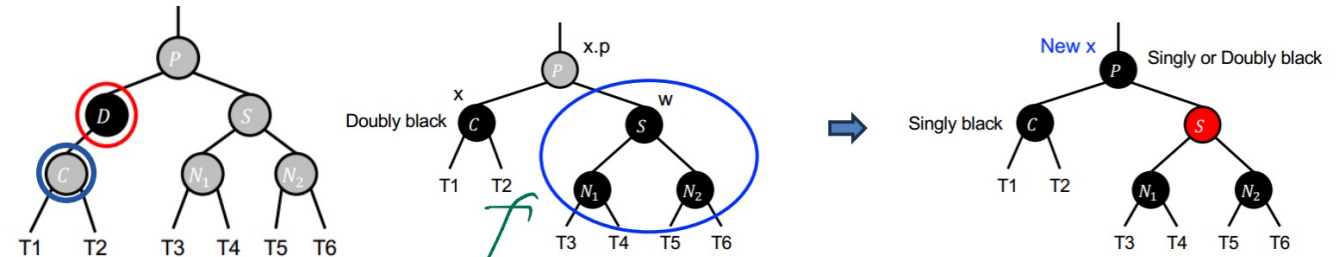
형제노드가 Red인 상황에서는 형제노드의 부모 P와 형제노드의 자식은 모두 Black이다.

부모 P를 기준으로 Left Rotation을 하고, P를 Red, S를 Black으로 칠한다.

Case 2-2 후계자가 Black이고 Sibling(형제)가 Black인데, Children이 Black일 때

-> Recoloring

Sibling을 Red로 색칠



C는 singly Black 노드가 되는데... 부모 노드가 문제.

부모 노드 P의 하위 서브트리 양쪽에서 블랙이 각각 1개씩 빠진 상태가 되었기 때문에, 이제는 부모 노드 P가 전체 트리 관점에서 블랙 1개가 부족한 책임을 떠안게 됩니다.

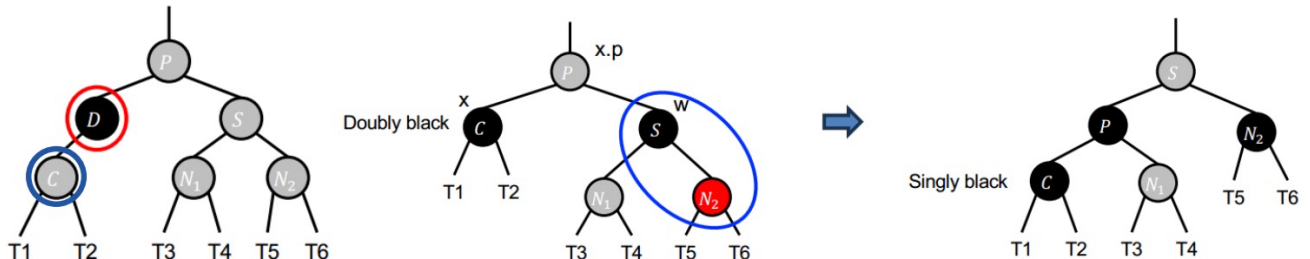
이 상황에서, 부모 노드 P가 new x가 되었다고 했을 때, 이 색깔에 따라 진행 방식이 달라진다.

부모가 원래 Red 였으면 -> 부모를 Black으로 채워주고, 그대로 Singly Black으로 종료

부모가 원래 Black이었으면 -> 다시 Doubly Black 상태. 해결해야 함.

Case 2-3 후계자가 Black이고 Sibling(형제)가 Black인데, Children이 Red일 때

-> Left Rotation + Recoloring (P와 N2가 Black)



현재 노드가 삭제된 후 왼쪽 자식인 C가 위로 올라왔으나, BLACK 노드라 여전히 Doubly Black 상태입니다.

* Doubly Black : 아래 자식이 삭제돼서 내가 Black 2개치의 부담을 짊어지고 있음.

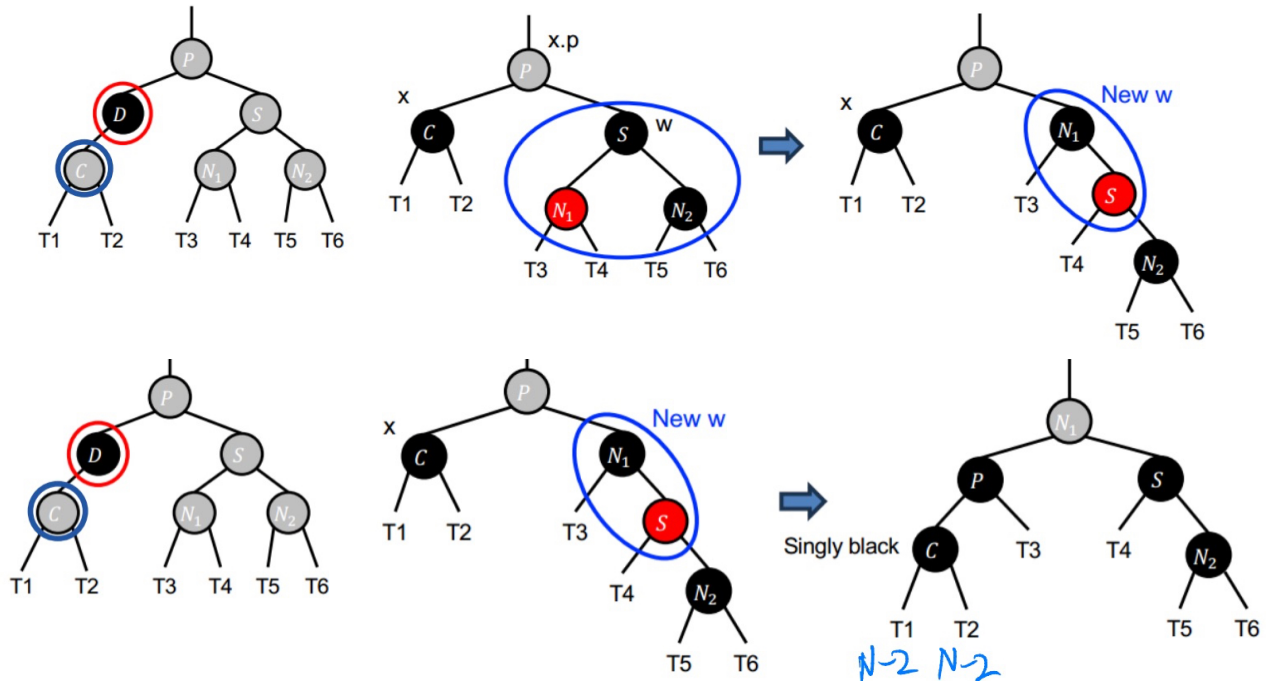
즉, 왼쪽 경로의 블랙 높이가 1 부족

오른쪽 형제 노드 S의 색상은 BLACK

형제의 오른쪽 자식인 **N2**가 **RED**입니다. (이때 왼쪽 자식인 **N1**의 원래 색상은 무엇이든 상관없음.
 부모 노드 **P**를 기준으로 트리를 **Left Rotation**
 원래 **RED**였던 오른쪽 조카 **N2**를 **BLACK**으로 바꿈.
 형제 노드 **S**는 원래 부모(**P**)가 가지고 있던 색상을 그대로 물려받습니다.

오른쪽에서 왼쪽으로 블랙 노드를 넘겨주어(Pass over) 부족한 블랙 높이를 보상(Compensate)!

Case 2-4 후계자가 **Black**이고 Sibling(형제)가 **Black**인데, Left child는 **Red**이고, Right Child는 **Black**일 때
 Case 2-3(오른쪽 조카가 **RED**인 상황)으로 강제 변환하는 것이 이 슬라이드의 핵심 목표



C는 여전히 **Doubly Black** 상태로, 왼쪽 경로의 블랙 높이가 1 부족

S의 색상은 **Black**, 왼쪽 조카 **N1**은 **RED**, 오른쪽 조카 **N2**는 **BLACK**으로 비대칭.

-> **Right Rotation**해서 일렬로 줄세운다. 그런 다음, 왼쪽 조카 **N1**은 **BLACK**으로, 아래로 내려가는 원래 형제 **S**는 **RED**로 색상을 바꿉니다. 검-빨-검 이렇게 되게.

그 다음부터는 새로운 형제가 **BLACK**이고 그의 오른쪽 자식이 **RED**인 상태는, 우리가 바로 직전에 마스터했던 **[CASE 2-3]**의 조건과 완벽하게 일치 -> 2-3 조건을 해서 마무리 해야한다!

Red-Black Tree example

Delete(15)

-> Delete(33)

-> Delete(37)

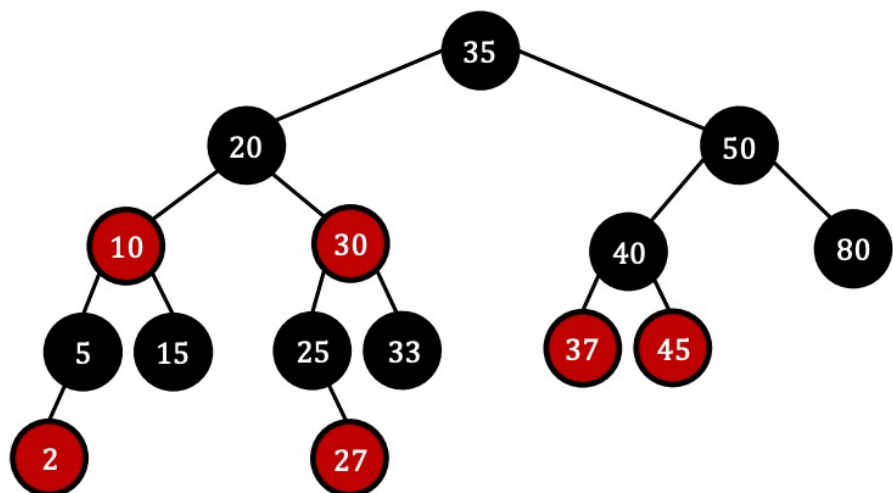
-> Delete(35)

-> Delete(40)

-> Delete(50)

-> Delete(80)

-> Delete(27)



1. Delete(15)

15번 노드(검은색 단말 노드)를 삭제하면서 해당 빈자리에 이중 흑색(Double Black) 노드(NIL)가 발생합니다.

적용 케이스: [CASE 2-4] (형제 노드가 Black + 바깥쪽 조카가 Red)

삭제된 15의 형제 노드(5)는 검은색이고, 형제 노드의 바깥쪽 자식(2)이 빨간색인 상황입니다.

어떻게 해야 하는가: 형제 노드 방향으로 부모 노드(10)를 기준으로 우회전(Right Rotation)을 수행합니다. 이후 기존 부모 노드와 자식 노드들의 색상을 변경(Recoloring)하여 이중 흑색 상태를 해소하고 균형을 맞춥니다.

2. Delete(33)

33번 노드(검은색 단말 노드)를 삭제하면서 다시 빈자리에 이중 흑색(Double Black)이 발생합니다.

적용 케이스: [CASE 2-3] (형제 노드가 Black + 안쪽 조카가 Red)

형제 노드(25)는 검은색이고, 형제 노드의 안쪽 자식(27)이 빨간색인 상황입니다.

어떻게 해야 하는가: 먼저 형제 노드(25)를 기준으로 좌회전(Left Rotation)을 수행하여 안쪽 조카(27)를 바깥쪽으로 빼냅니다. 이 과정을 통해 트리를 [CASE 2-4] 상태로 변환시킵니다. 이후 부모 노드(30)를 기준으로 우회전(Right Rotation)을 수행하고 색상을 변경하여 이중 흑색을 완전히 해소합니다. (결과적으로 27이 부모 위치로 올라옵니다.)

3. Delete(37)

37번 노드는 빨간색(Red) 단말 노드입니다.

적용 케이스: 해당 없음 (단순 삭제)

어떻게 해야 하는가: 빨간색 노드는 경로의 검은색 노드 수(Black Height)에 영향을 주지 않으므로, 복잡한 Case 확인이나 회전 작업 없이 단순히 노드를 제거하면 완료됩니다.

4. Delete(35)

자식이 2개인 35번 노드를 삭제합니다.

적용 케이스: 자식이 2개인 노드의 삭제 원리 적용

어떻게 해야 하는가: 오른쪽 서브트리에서 가장 작은 값인 후계자(Successor) 노드 40을 찾아 35번의 자리로 값만 복사하여 교체합니다. 그 후, 기존에 40번이 있던 원래 자리를 삭제합니다. (이때 삭제되는 원래 40번 자리는 빨간색 취급되어 추가적인 회전 없이 단순히 지워집니다.)

5. Delete(40)

방금 35번 자리를 대체하고 올라갔던 40번 노드를 다시 삭제합니다. 이번엔 40의 후계자인 45번 노드가 그 자리를 대체하기 위해 올라옵니다.

적용 케이스: [CASE 1] (Successor C is Red / 자식이 1개인 경우)

어떻게 해야 하는가: 대체하기 위해 올라온 원래의 45번 노드 자리를 지워야 하는데, 45번에게는 50번이라는 자식이 하나 있습니다. 이 경우 45번 위치를 자식인 50번 노드로 바로 연결(대체)해 줍니다. 대체하고 올라온 50번 노드가 빨간색이므로 색상을 검은색(Black)으로 칠하여(Recoloring) 이중 흑색 문제를 간단히 해결합니다.

6. Delete(50)

50번 노드(빨간색 단말 노드)를 삭제합니다.

적용 케이스: 해당 없음 (단순 삭제)

어떻게 해야 하는가: 37번 노드를 삭제했을 때와 동일하게, 빨간색 단말 노드이므로 흑색 높이 규칙을 위반하지 않습니다. 회전 없이 단순히 제거합니다.

7. Delete(80)

80번 노드(검은색 단말 노드)를 삭제하며 빈자리에 이중 흑색 노드가 생깁니다. 이 단계는 연쇄 반응을 일으킵니다.

1차 적용 케이스: [CASE 2-2] (형제가 Black + 두 조카 모두 Black)

80의 형제가 되는 NIL 노드들은 모두 Black입니다.

어떻게 해야 하는가: 형제 노드의 색상을 빨간색으로 변경(Recoloring)하고, 이중 흑색(Double Black) 상태를 위쪽(부모 노드)으로 밀어 올립니다.

2차 적용 케이스: [CASE 2-1] (형제가 Red)

위로 밀려 올라간 이중 흑색 노드의 새로운 형제 노드가 빨간색인 상황이 됩니다.

어떻게 해야 하는가: 부모 노드를 기준으로 회전(Rotation) 및 색상 변경(Recoloring)을 수행합니다. 이 작업 자체로 이중 흑색이 당장 사라지는 것은 아니지만, 트리의 구조를 Case 2-2, 2-3, 2-4 중 하나로 변환시켜 줍니다. 이후 변환된 케이스에 맞춰 추가 작업을 진행하여 균형을 맞춥니다.

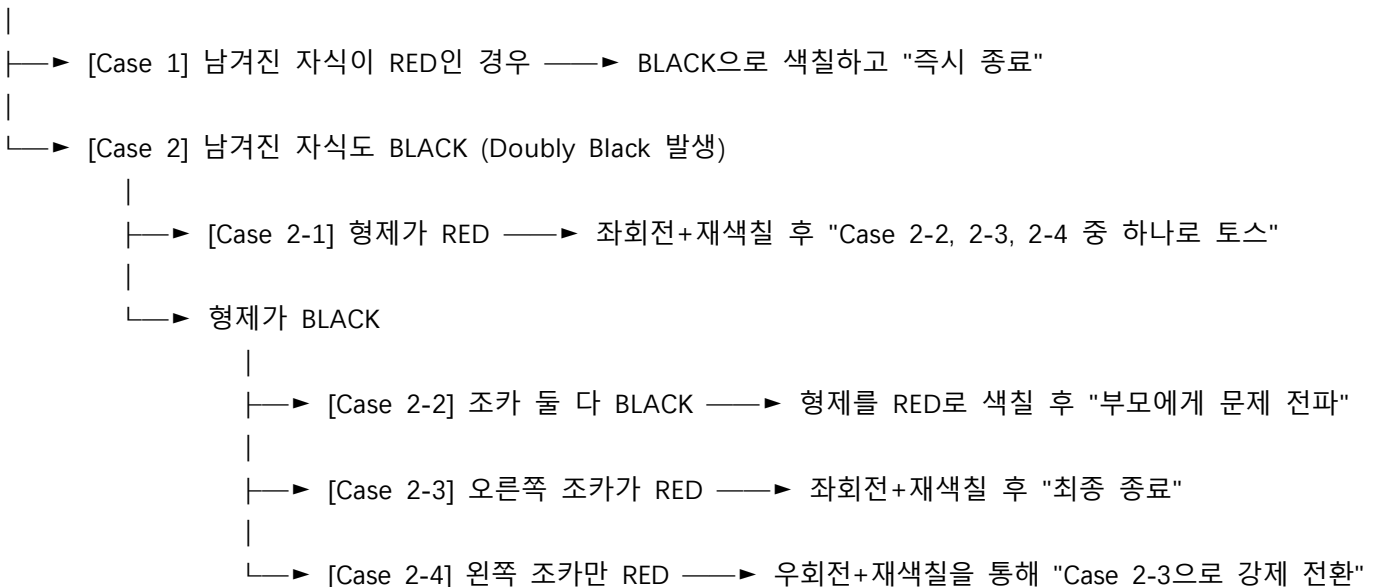
8. Delete(27)

자식이 2개인 27번 노드(루트 근처)를 삭제합니다.

적용 케이스: 자식이 2개인 노드의 삭제 원리 적용

어떻게 해야 하는가: 27번의 후계자(Successor)인 30번 노드를 찾아 27번의 자리로 값을 복사해 교체합니다. 그 후 기존의 30번 노드를 제거합니다. 지워지는 30번 노드는 빨간색 단말 노드였기 때문에, 추가적인 회전이나 속성 위반 없이 단순 제거하는 것으로 모든 실습 예제가 종료됩니다.

[삭제 대상 노드가 BLACK일 때]



Priority Queue

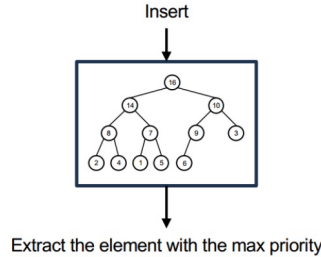
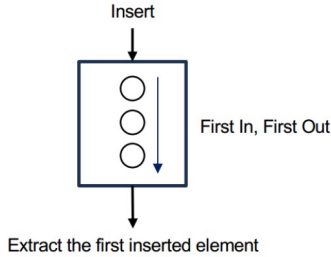
Definition

A type of queue that arranges elements based on their **priority**

Key operations: **Insert(x, Q)**, **getMax(Q)**, **getMin(Q)**, **deleteMax(Q)**, **deleteMin(Q)**

Ordinary Queue: 우리가 잘 아는 선입선출(FIFO, First-In-First-Out) 방식. 먼저 들어온 데이터가 먼저 처리됨.

Priority Queue: 삽입된 순서와 처리되는 순서가 일치하지 않습니다 (**Order in elements inserted $\neq$ Order in elements processed**). 들어온 순서가 아니라, 데이터가 가지는 '**우선순위**'에 따라 먼저 처리될 대상이 결정



Priority Queue can be implemented with various data structures : Array(ordered/unordered), Linked List, Binary Heap

Binary Heap

: A Binary Heap is a **Complete Binary Tree** that **satisfies the Heap Property**

Complete Binary Tree

마지막 레벨을 제외한 모든 레벨의 노드가 완전히 채워져 있어야 합니다.

마지막 레벨의 노드들은 최대한 왼쪽부터 빈틈없이 채워져야 합니다.

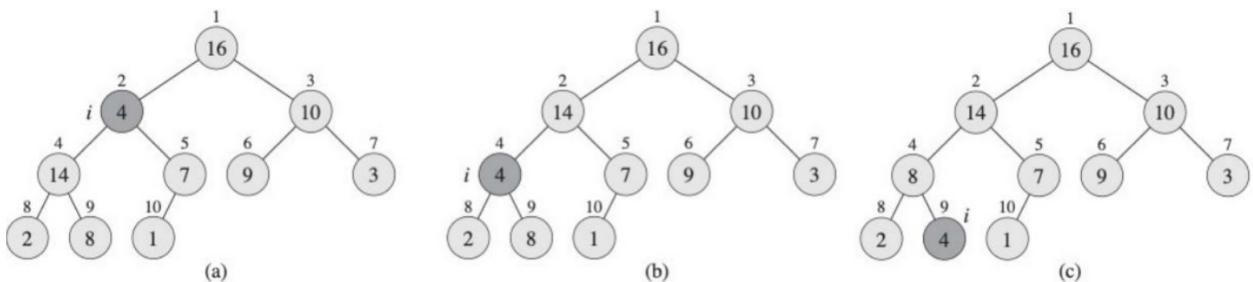
Heap Property

Max-Heap: 부모 노드의 값이 자식 노드의 값보다 크거나 같아야 합니다. 이는 트리 전체에서 가장 큰 값이 항상 루트 노드에 위치하도록 보장합니다.

Min-Heap: 부모 노드의 값이 자식 노드의 값보다 작거나 같아야 합니다. 이는 트리 전체에서 가장 작은 값이 항상 루트 노드에 위치하도록 보장합니다.

★힙(Heap)이 특정 데이터를 탐색(Search)하는 데는 왜 부적합(useless)★

부모-자식간의 관계만 대소를 나타낼 뿐이지 왼쪽 자식, 오른쪽 자식은 알 수가 없다.



MAX-HEAPIFY(A, i)

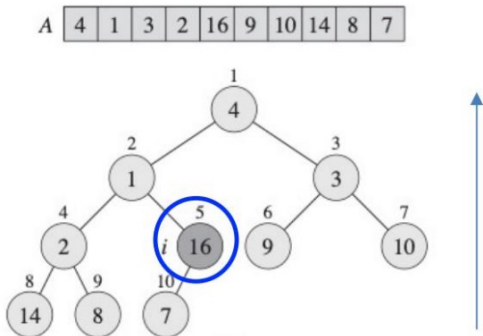
```
1 l = LEFT(i)
2 r = RIGHT(i)
3 if l ≤ A.heap-size and A[l] > A[i]
4     largest = l
5 else largest = i
6 if r ≤ A.heap-size and A[r] > A[largest]
7     largest = r
8 if largest ≠ i
9     exchange A[i] with A[largest]
10    MAX-HEAPIFY(A, largest)
```

Max-Heap 속성 위반이면, 자식 중 더 큰 것을 찾아서 SWAP
아래로 내려가면서 계속하기

Building a Max Binary Heap

- Since the Binary Heap is a complete binary tree, it can be written as an array
- Start from the **leaf node** and **move up** 아래에서 위로 올라온다.

- Let say we have the elements written in random order



시험에서는 이렇게 무작위 Array가 나온다!

BUILD-MAX-HEAP(A)

```

1  A.heap-size = A.length
2  for i = [A.length/2] downto 1
3      MAX-HEAPIFY(A, i)
    
```

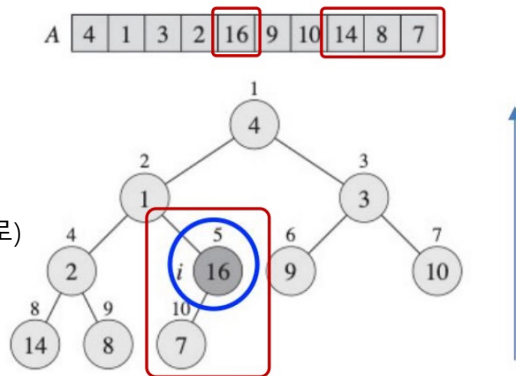
BUILD-MAX-HEAP(A)

```

1  A.heap-size = A.length
2  for i = [A.length/2] downto 1
3      MAX-HEAPIFY(A, i)
    
```

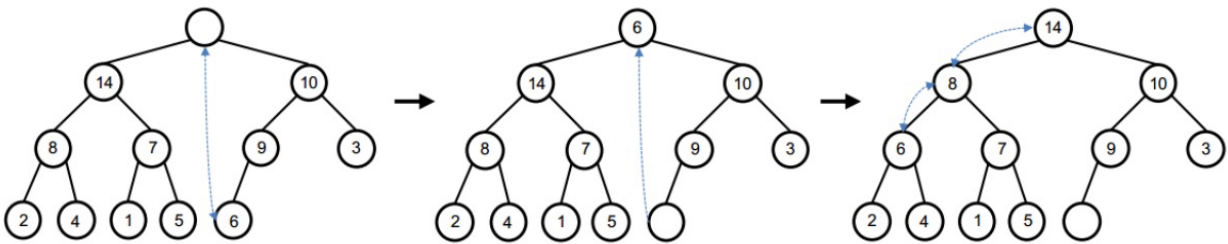
반평해서 내림한 노드부터 Heapify 시작.
그 노드부터 점점 루트로 올라오기(배열 한칸씩 앞으로)

- Let say we have the elements written in random order



Extract Max (Deletion) Max 삭제

- 1단계: 루트 노드 추출 및 '맨 마지막 노드' 올리기 (첫 번째 그림) Extract Element with the Highest Priority
- 2단계: 임시 안착 및 규칙 위반 확인 (두 번째 그림)
- 3단계: **MAX-HEAPIFY**로 수리하기 (세 번째 그림) -> 루트 노드 Max-Heapify하기

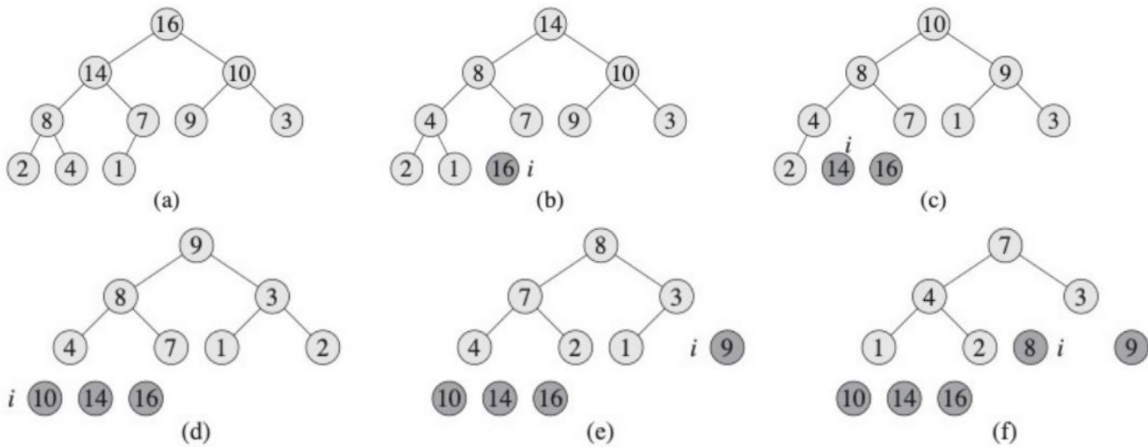


만약 맨 마지막 노드 말고 다른 자식 노드를 뽑아서 하면 어떻게 될까? -> 더 이상 Complete Binary Tree가 아니게 된다.

그러면 맨 마지막 노드를 그 빈자리에 채우면 되는거 아냐? -> **Heap 속성이 깨진다.**

Heap Sort

1. Build a Max-Heap
정렬되지 않은 무작위 배열을 바텀업 방식으로 힙 파일링하여 **완벽한 최대 힙으로 만듭니다.**
2. Exchange & Extract
맨 위에 있는 가장 큰 값 16을 힙의 맨 마지막 노드인 1과 자리를 바꿉니다(Exchange).
16은 배열의 맨 마지막 칸(올바른 정렬 위치)에 정착했으므로 더 이상 힙 연산에 참여시키지 않음.
3. Max-Heapify로 다음 대장 선출
맨 위로 뽑아 왔으니 Heapify를 해서 Heap 속성 만족시키도록 해야 함.
그다음 2-3단계 반복!



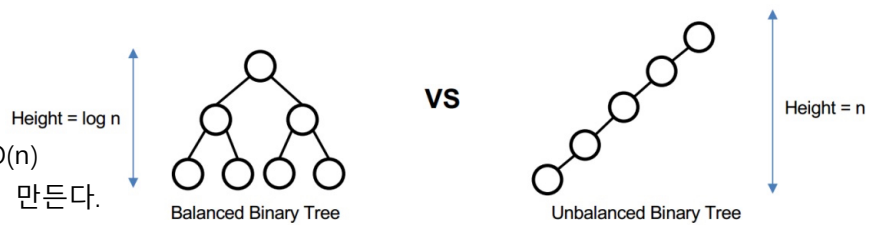
OX 대비 : **Max-Heap** 가장 큰 놈을 뽑아서 **배열의 맨 뒤부터** 채워 넣기 때문에, 최종 결과는 **Ascending Order** 오름차순
Min-Heap 가장 작은놈을 뽑아서 **배열의 맨 뒤부터** 채워 넣기 때문에, 최종 결과는 **Descending Order** 내림차순

Build-Max-Heap	$O(n)$
MAX-HEAPIFY	$O(\log n)$
Heap Sort	$O(n \log n)$

AVL Tree

Binary Search Tree의 단점

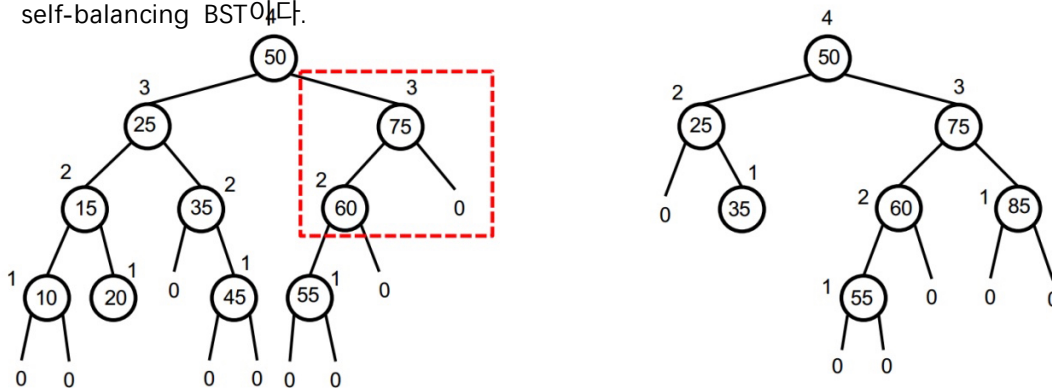
height에 따라서 퍼포먼스가 달라짐
 best case는 $O(\log n)$, worst case는 $O(n)$
 BAD한 시퀀스는 BST를 언밸런스하게 만든다.



Adelson-Velsky and Landis (AVL) Tree

A self-balancing BST in which the difference between the heights of left and right subtrees of any nodes is at most 1, ensuring logarithmic tree height.

임의의 node들의 **left subtree**와 **right subtree**의 **height** 차이가 최대 1이며, logarithmic tree height를 보장하는 self-balancing BST이다.



How can we check if a (sub)tree is balanced or not?

→ **Balance Factor**: the difference between the heights of its left and right subtrees

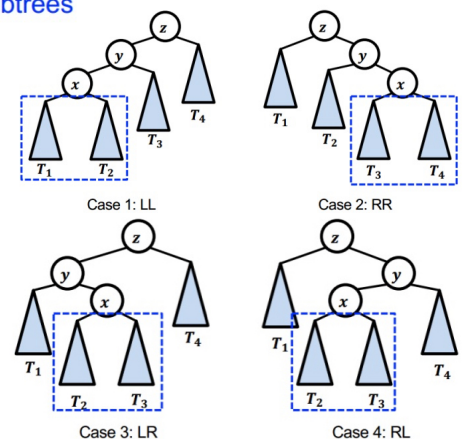
(sub)tree가 balanced 상태인지 아닌지 어떻게 확인할 수 있는가?

Balance Factor: **left subtree**와 **right subtree**의 **height** 차이

Insertion

■ Four cases when node z makes T unbalanced

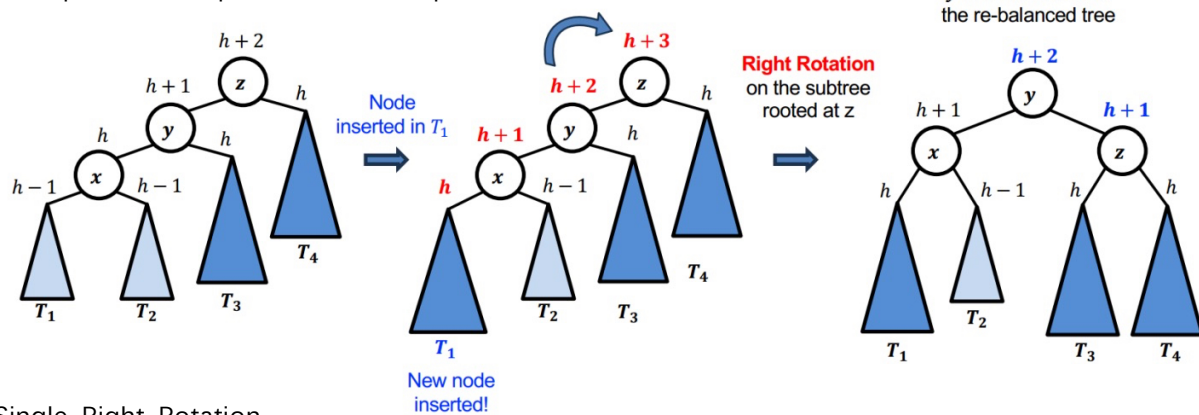
- [Case 1] (LL) : Insertion into the **Left** subtree of the **Left** child of node z .
- [Case 2] (RR) : Insertion into the **Right** subtree of the **Right** child of node z .
- [Case 3] (LR) : Insertion into the **Right** subtree of the **Left** child of node z .
- [Case 4] (RL) : Insertion into the **Left** subtree of the **Right** child of node z .



Case 1 LL

Insertion into the Left subtree of the Left child of node z.

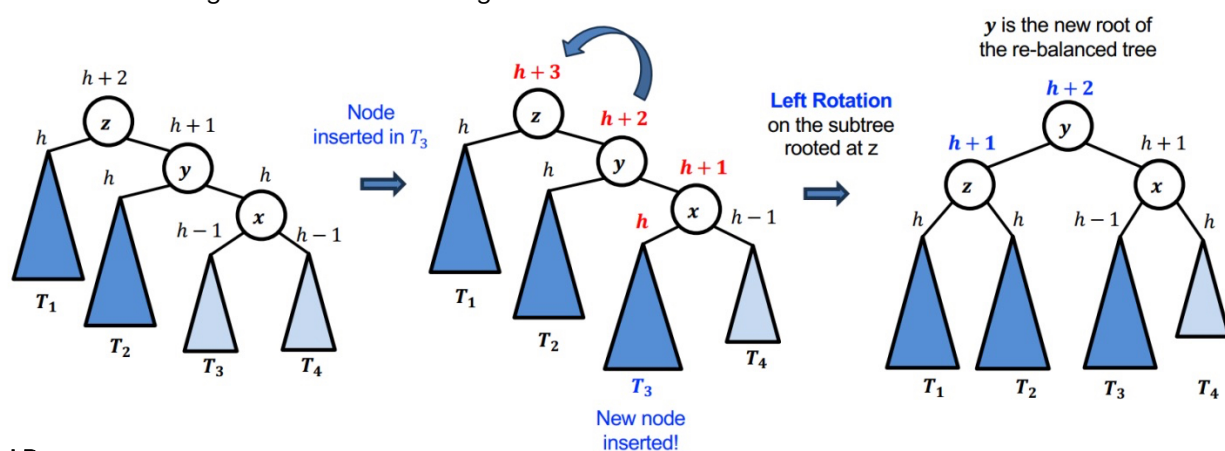
Node z의 Left child의 Left subtree로의 Insertion.



->Single Right Rotation

Case 2 RR

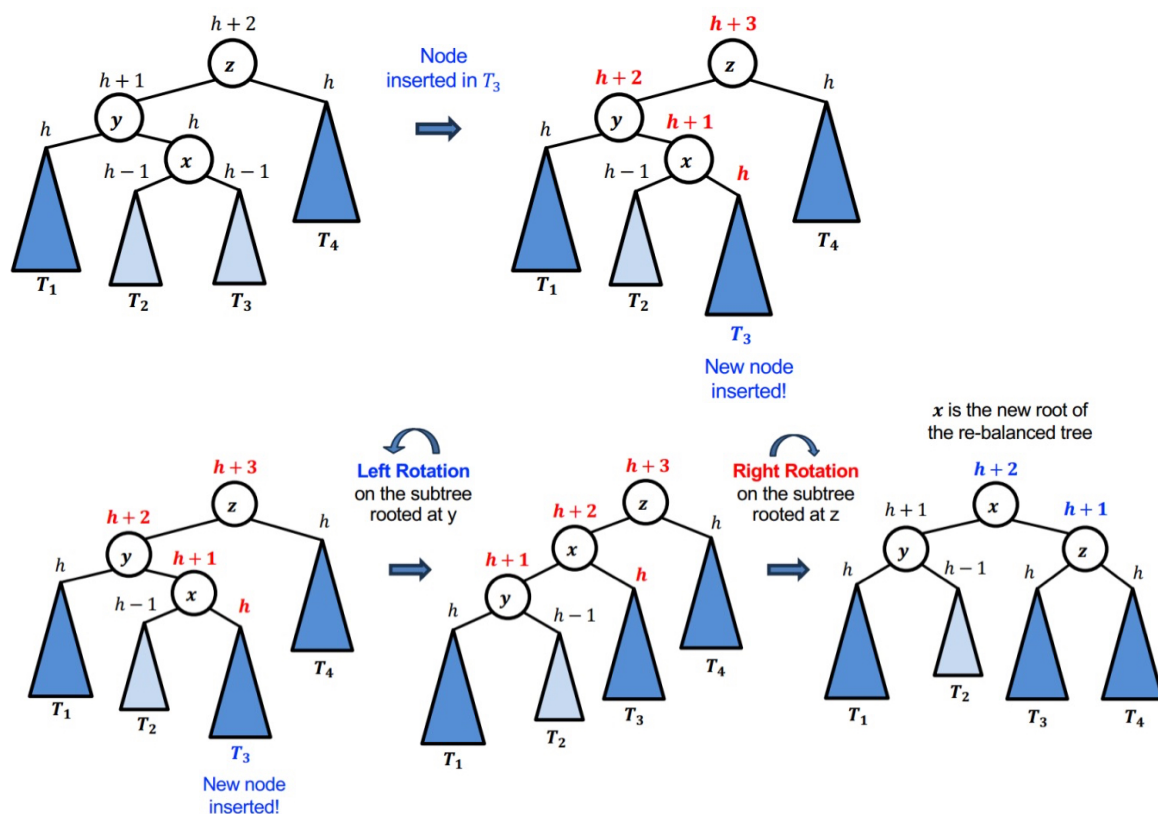
Insertion into the Right subtree of the Right child of node z.



Case 3 LR

Insertion into the Right subtree of the Left child of node z.

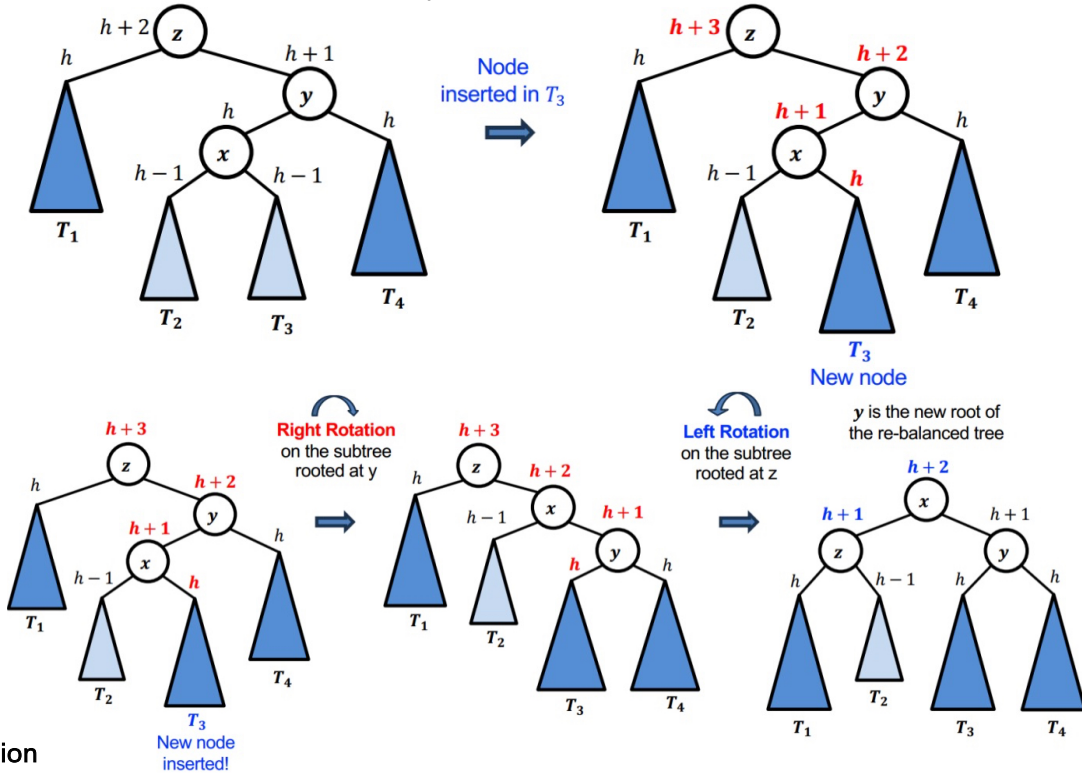
Double Rotation: Left Rotation (y) + Right Rotation (z)



Case 4 RL

Insertion into the Left subtree of the Right child of node z

- Double Rotation: Right Rotation (y) + Left Rotation (z)



Deletion

- [1] Perform standard BST deletion for the key w
 - (1) if the node has **no** child, (2) if the node has **one** child, (3) if the node has **two** children
- [2] Starting with w , find the first unbalanced node (i.e., one of the ancestors)
 - Let z be the first unbalanced node, y be the larger child of z and x be the larger child of y
- [3] Re-balance the tree by performing appropriate rotations on the subtree rooted at the z , the unbalanced node.

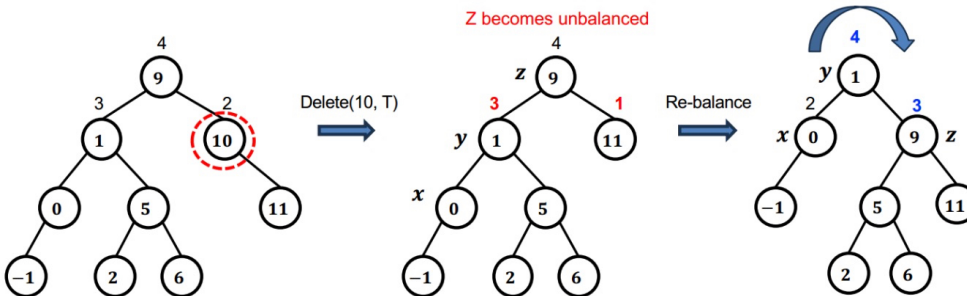
1. Key w 에 대해 표준 BST deletion을 수행한다.

- (1) 만약 node에 child가 없는 경우, (2) 만약 node에 하나의 child가 있는 경우, (3) 만약 node에 두 개의 child가 있는 경우

2. w 에서 시작하여, 첫 번째 unbalanced node(즉, ancestor 중 하나)를 찾는다.

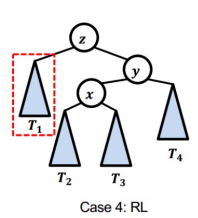
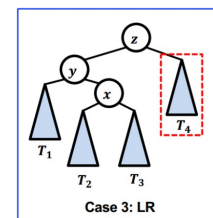
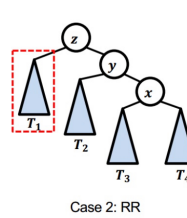
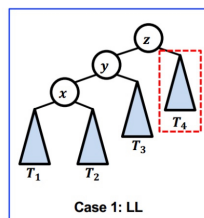
z 를 첫 번째 unbalanced node라 하고, y 를 z 의 더 큰 child, x 를 y 의 더 큰 child라고 하자.

3. unbalanced node인 z 를 root로 하는 subtree에 적절한 rotation을 수행하여 tree를 Re-balance한다.



Deletion에는 4가지가 있다

- Deletion of a node in T_4 : Case 1 (LL), Case 3 (LR)
 - When the height of T_4 decreases.
- Deletion of a node in T_1 : Case 2 (RR), Case 2 (RL)
 - When the height of T_1 decreases



Cases 1 and 3 are symmetric to Cases 2 and 4

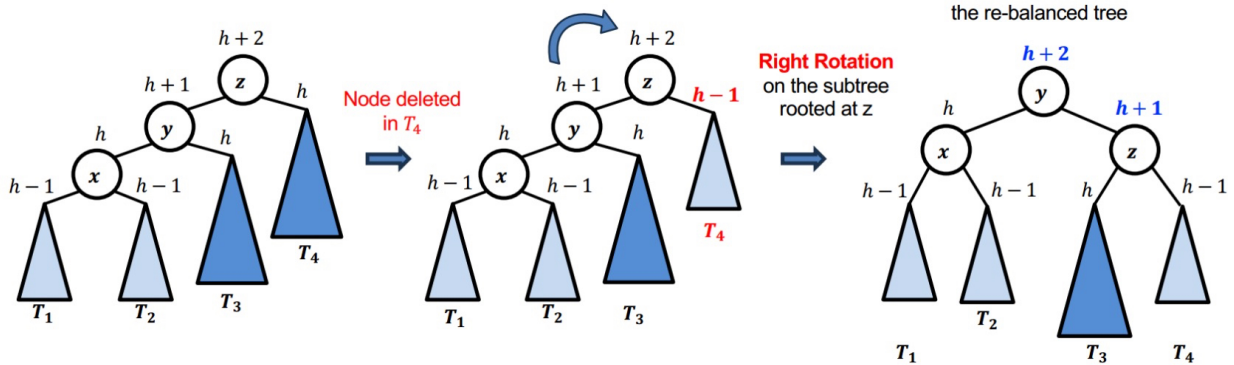
Case 1

T4 지우기

Case 2랑 대칭!

= Insertion into the Right subtree of the Left child of node z

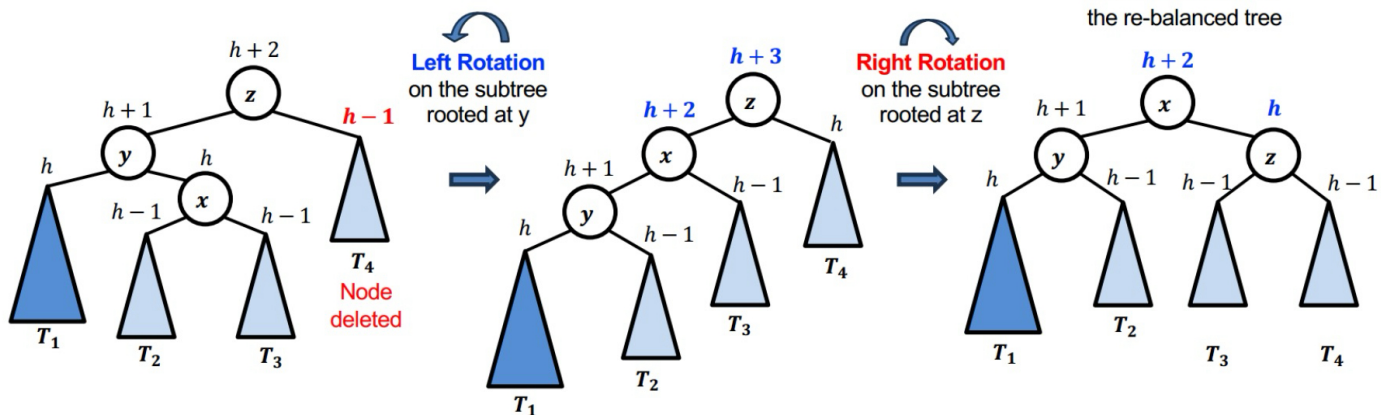
LR case (Double rotation) : Left rotation(y) + Right rotation (z)



Case 3

T4 지우기

Case 4랑 대칭!



example

Insert(50), Insert(70), Insert(90), Insert(20), Insert(60), Insert(30), Delete(20), Insert(10),
 Insert(55), Insert(65), Delete(30), Insert(57), Delete(90), Delete(60), Delete(57), Delete(70),
 Insert(20)

week13-18 ~

Hash Table

Previous: Direct Address Tables

많은 응용 프로그램들은 오직 Dictionary operation(INSERT, SEARCH, DELETE)만을 지원하는 Dynamic set을 필요로 한다.

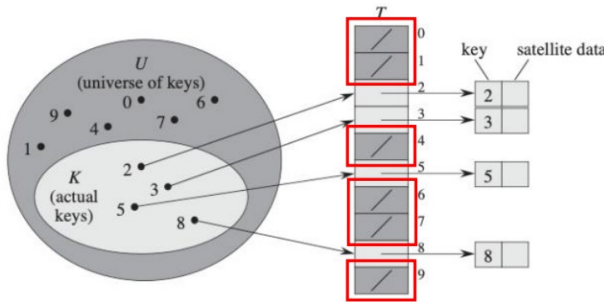
Key k 를 Slot k 에 저장한다.

한계점: 각 원소는 반드시 **Distinct key**를 가져야 한다

만약 $|K|/|U|$ 가 작다면, 대부분의 Slot은 낭비된다.

더욱이, 컴퓨터의 제한된 메모리를 고려할 때 크기가 $|U|$ 인 Table T 를 저장하는 것은 비실용적(심지어 불가능)일 수 있다.

어떻게 하면 Time complexity $O(1)$ 을 보존하면서 Space complexity를 $O(|K|)$ 로 줄일 수 있을까?



```

DIRECT-ADDRESS-SEARCH( $T, k$ )
1 return  $T[k]$   $O(1)$ 

DIRECT-ADDRESS-INSERT( $T, x$ )
1  $T[x.key] = x$   $O(1)$ 

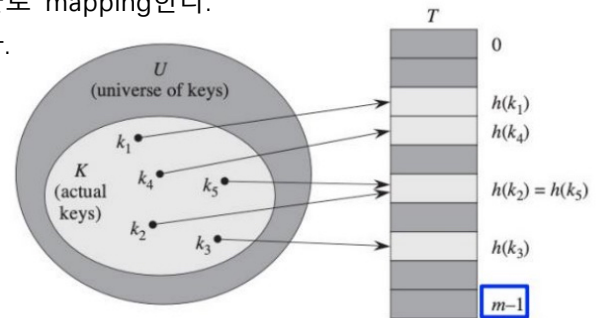
DIRECT-ADDRESS-DELETE( $T, x$ )
1  $T[x.key] = \text{NIL}$   $O(1)$ 
    
```

What is hashing? (해싱이란 무엇인가?)

Hash function h 를 사용하여 key k 를 slot k 가 아니라 slot $h(k)$ 에 저장하는 것이다.

Hash function h 는 universe of keys U 를 hash table T 의 slot들로 mapping한다.

따라서, space requirement는 hash table T 의 size와 동일해진다.



그러나, Hash collision은 필연적으로 발생한다

- Two keys k_1 and k_2 where $k_1 \neq k_2$ but $h(k_1) = h(k_2)$ under hash function h
- Two keys k_1 and k_2 where $k_1 \neq k_2$, $h(k_1) \neq h(k_2)$, but $h(k_1) \% m = h(k_2) \% m$
- Avoiding collisions is impossible when $|U| > m$

Collision resolution by Chaining

■ Insertion: $O(1)$

■ Deletion: $O(1)$

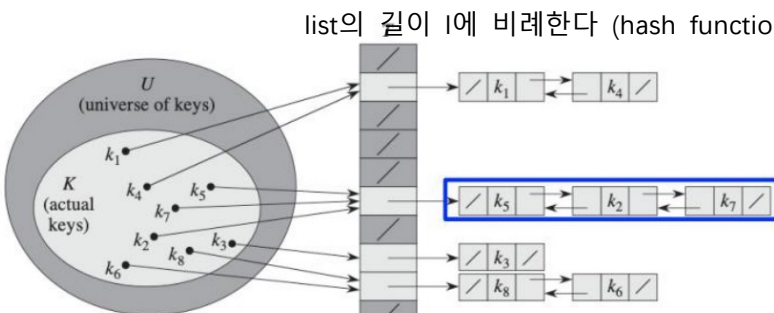
□ It takes an element x as input, not its key k (we do not need to search for x)

이것은 key k 가 아니라 element x (노드의 메모리 주소를 받는걸 전제)를 input으로 받는다

■ Search: $O(l)$

(우리는 x 를 search할 필요가 없다).

□ Proportional to the length of the list l (depending on a hash function)



list의 길이 l 에 비례한다 (hash function에 의존함).

```

CHAINED-HASH-INSERT( $T, x$ )
1 insert  $x$  at the head of list  $T[h(x.key)]$ 

CHAINED-HASH-SEARCH( $T, k$ )
1 search for an element with key  $k$  in list  $T[h(k)]$ 

CHAINED-HASH-DELETE( $T, x$ )
1 delete  $x$  from the list  $T[h(x.key)]$ 
    
```

Worst-case complexity: $O(n)$

Average-case complexity: $O(1 + \alpha)$ (hash function을 계산하는 것 + 탐색하는 것)

Load factor: $\alpha = n/m$ (chain의 평균 길이) n 은 element개수 m 은 테이블 slot 개수

이 분석은 무엇을 의미하는가? -> Search operation은 $O(1)$ time에 실행될 수 있다

만약 좋은 hash function을 사용하여 데이터가 table 전체에 균등하게 잘 분산되고, table의 크기 m 이 element의 개수 n 에 비례하여 충분히 크다면 Load factor α 는 작은 상수로 유지된다. 결과적으로 평균적인 상황에서 Search operation을 상수 시간인 $O(1)$ 에 빠르게 수행할 수 있다

$m = 13,$
 $k = \{5, 14, 29, 25, 17, 21, 18, 32, 20, 9, 15, 27\}$
 $h(k) = k \bmod 13$

HASH-INSERT(T, k)

```
1 i = 0
2 repeat
3   j = h(k, i)
4   if T[j] == NIL
5     T[j] = k
6   return j
7   else i = i + 1
8 until i == m
9 error "hash table overflow"
```

HASH-SEARCH(T, k)

```
1 i = 0
2 repeat
3   j = h(k, i)
4   if T[j] == k
5     return j
6   i = i + 1
7 until T[j] == NIL or i == m
8 return NIL
```

0		
1	14	27
2	15	
3	29	
4	17	
5	5	18
6		32
7		20
8	21	
9		9
10		
11		
12	25	

Open Addressing에 의한 collision resolution(충돌 해결)

모든 element를 hash table 자체에 저장하는 것

각 slot은 하나의 key 또는 NIL을 포함한다.

장점 : pointer가 필요하지 않다.

여분의 memory는 hash table에게 더 많은 수의 slot을 제공한다. 더 적은 collision, 더 빠른 retrieval.

비어있는 slot을 찾거나(insert) key를 찾을 때까지(search) hash table을 조사(probe)한다.

모든 key k 에 대한 probe sequence -> $h(k, 0), h(k, 1), \dots, h(k, m-1)$

3가지 방법 Linear Probing / Quadratic Probing / Double Hashing

Linear Probing

$$h': U \rightarrow \{0, 1, \dots, m-1\}$$

$$h(k, i) = (h'(k) + i) \bmod m$$

이 method는 primary clustering으로 알려진 문제점을 겪는다:

차지된 slot들이 긴 연속(long runs)을 형성하며 쌓이게 되고, 평균적인 search time을 증가시킨다.

HASH-INSERT(T, k)

```
1 i = 0
2 repeat
3   j = h(k, i)
4   if T[j] == NIL
5     T[j] = k
6   return j
7   else i = i + 1
8 until i == m
9 error "hash table overflow"
```

HASH-SEARCH(T, k)

```
1 i = 0
2 repeat
3   j = h(k, i)
4   if T[j] == k
5     return j
6   i = i + 1
7 until T[j] == NIL or i == m
8 return NIL
```

$m = 13,$
 $k = \{5, 14, 29, 25, 17, 21, 18, 32, 20, 9, 15, 27\}$
 $h(k) = k \bmod 13$

0		
1	14	27
2	15	
3	29	
4	17	
5	5	18
6		32
7		20
8	21	
9		9
10		
11		
12	25	

$m = 13,$
 $k = \{5, 14, 29, 25, 17, 21, 18, 32, 20, 9, 15, 27\}$
 $h(k) = k \bmod 13$
 $h(k, i) = (k + i) \bmod 13$

This method suffers from a problem known as **primary clustering**:
Long runs of occupied slots build up, increasing the average search time

Quadratic Probing

$$h': U \rightarrow \{0, 1, \dots, m-1\}$$

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

HASH-INSERT(T, k)

```

1  i = 0
2  repeat
3    j = h(k, i)
4    if T[j] == NIL
5      T[j] = k
6      return j
7    else i = i + 1
8  until i == m
9  error "hash table overflow"
```

HASH-SEARCH(T, k)

```

1  i = 0
2  repeat
3    j = h(k, i)
4    if T[j] == k
5      return j
6    i = i + 1
7  until T[j] == NIL or i == m
8  return NIL
```

$m = 13$,
 $k = \{5, 14, 29, 25, 17, 21, 18, 32, 20, 9, 15, 27\}$
 $h(k) = k \bmod 13$

0		
1	14	27
2	15	
3	29	
4	17	
5	5	18
6	32	
7	20	
8	21	
9		9
10		
11		
12	25	

$m = 13$,
 $k = \{5, 14, 29, 25, 17, 21, 18, 32, 20, 9, 15, 27\}$
 $h(k) = k \bmod 13$
 $h(k, i) = (k + i + 3i^2) \bmod 13$

Jump to avoid primary clustering

However, it still have a problem known as **secondary clustering**:
 If two keys have the same h' value, then they have the same probe sequence

그러나, 여전히 **secondary clustering**으로 알려진 문제점을 가지고 있다:

만약 두 key가 동일한 h' 값을 가진다면, 그들은 동일한 probe sequence를 가진다.

Double Hashing

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

HASH-INSERT(T, k)

```

1  i = 0
2  repeat
3    j = h(k, i)
4    if T[j] == NIL
5      T[j] = k
6      return j
7    else i = i + 1
8  until i == m
9  error "hash table overflow"
```

HASH-SEARCH(T, k)

```

1  i = 0
2  repeat
3    j = h(k, i)
4    if T[j] == k
5      return j
6    i = i + 1
7  until T[j] == NIL or i == m
8  return NIL
```

$m = 13$,
 $k = \{5, 14, 29, 25, 17, 21, 18, 32, 20, 9, 15, 27\}$
 $h(k) = k \bmod 13$

0		
1	14	27
2	15	
3	29	
4	17	
5	5	18
6	32	
7	20	
8	21	
9	9	
10		
11		
12	25	

$m = 13$,
 $k = \{5, 14, 29, 25, 17, 21, 18, 32, 20, 9, 15, 27\}$
 $h_1(k) = k \bmod 13$
 $h_2(k) = 1 + (k \bmod 11)$
 $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod 13$

Hashing Example

- Consider inserting the keys 10, 22, 31, 4, 15, 28, 17, 88, 59 into a hash table of length $m=11$ using open addressing with the auxiliary hash function $h'(k) = k$. Illustrate the result of inserting these keys using (1) linear probing, (2) using quadratic probing with $c_1 = 1$ and $c_2 = 3$, and (3) double hashing with $h_1(k) = k$ and $h_2(k) = 1 + (k \bmod (m - 1))$

0 mod 11	22
1 mod 11	88
2 mod 11	
3 mod 11	
4 mod 11	4
5 mod 11	15
6 mod 11	28
7 mod 11	17
8 mod 11	59
9 mod 11	31
10 mod 11	10

0 mod 11	22
1 mod 11	
2 mod 11	
3 mod 11	17
4 mod 11	4
5 mod 11	
6 mod 11	28
7 mod 11	
8 mod 11	15
9 mod 11	31
10 mod 11	10

0 mod 11	22
1 mod 11	
2 mod 11	
3 mod 11	
4 mod 11	4
5 mod 11	15
6 mod 11	
7 mod 11	
8 mod 11	
9 mod 11	31
10 mod 11	10

Linear Probing

$10 \bmod 11 = 10$
 $22 \bmod 11 = 0$
 $31 \bmod 11 = 9$
 $4 \bmod 11 = 4$
 $15 \bmod 11 = 4$
 $28 \bmod 11 = 6$
 $17 \bmod 11 = 6$
 $88 \bmod 11 = 0$
 $59 \bmod 11 = 4$

Quadratic Probing

$10 \bmod 11 = 10$
 $22 \bmod 11 = 0$
 $31 \bmod 11 = 9$
 $4 \bmod 11 = 4$
 $15 \bmod 11 = 4$
 $15 \bmod 11 = 4$, probe with $i=1$, $h(15,1) = (4 + 1 + 3(1^2)) \bmod 11 = 8$
 $28 \bmod 11 = 6$
 $17 \bmod 11 = 6$
 $17 \bmod 11 = 6$, probe with $i=1$, $h(17,1) = (6 + 1 + 3(1^2)) \bmod 11 = 10$
 $17 \bmod 11 = 6$, probe with $i=2$, $h(17,2) = (6 + 2 + 3(2^2)) \bmod 11 = 9$
 $17 \bmod 11 = 6$, probe with $i=3$, $h(17,3) = (6 + 3 + 3(3^2)) \bmod 11 = 3$
 $88 \bmod 11 = 0$
 $88 \bmod 11 = 0$, probe with $i=1$, $h(88,1) = (0 + 1 + 3(1^2)) \bmod 11 = 4$
 $88 \bmod 11 = 0$, probe with $i=2$, $h(88,2) = (0 + 2 + 3(2^2)) \bmod 11 = 3$
 $88 \bmod 11 = 0$, probe with $i=3$, $h(88,3) = (0 + 3 + 3(3^2)) \bmod 11 = 8$
 $88 \bmod 11 = 0$, probe with $i=4$, $h(88,4) = (0 + 4 + 3(4^2)) \bmod 11 = 4$
 $88 \bmod 11 = 0$, probe with $i=5$, $h(88,5) = (0 + 5 + 3(5^2)) \bmod 11 = 9$
 $88 \bmod 11 = 0$, probe with $i=6$, $h(88,6) = (0 + 6 + 3(6^2)) \bmod 11 = 4$

Double Hashing

$10 \bmod 11 = 10$
 $22 \bmod 11 = 0$
 $31 \bmod 11 = 9$
 $4 \bmod 11 = 4$
 $15 \bmod 11 = 4$
 $15 \bmod 11 = 4$, probe with $i=1$, $h(15,1) = (4 + 1 * (1 + 15 \bmod 10)) \bmod 11 = 10 \bmod 11 = 10$
 $15 \bmod 11 = 4$, probe with $i=2$, $h(15,2) = (4 + 2 * (1 + 15 \bmod 10)) \bmod 11 = 16 \bmod 11 = 5$